

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

PRÁCTICA EXPERIMENTAL 4 – UNIDAD IV

MATERIA:

INGENIERÍA DE REQUERIMIENTOS

EQUIPO C

INTEGRANTES:

GUTIERREZ ORTEGA GENESIS ADRIANA CI: 1250274477 - ggutierrez@uteq.edu.ec

NIEVES SANCHEZ JIMMY SAMUEL CI: 1250907878 - jnievess@uteq.edu.ec

CURSO:

CUARTO SEMESTRE «B»

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

REPOSITORIO GITHUB:

<https://github.com/GenessiGutierrez/ISR401-PFC-ERS-GrupoC.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Resumen	5
2.	Objetivos	6
2.1.	Objetivo general	6
2.2.	Objetivos específicos	6
3.	Fundamento teórico	7
3.1.	Marco normativo de la validación y la gestión de requisitos	7
3.2.	Validación de requisitos	7
3.2.1.	Verificación frente a validación	7
3.2.2.	Técnicas de validación	8
3.2.3.	Fases y roles de la inspección de Fagan	8
3.2.4.	Métricas de inspección	9
3.2.5.	Caso aplicado	9
3.3.	Gestión de requisitos	9
3.3.1.	Gestión de la configuración del ERS y línea base	9
3.3.2.	Atributos de los requisitos	9
3.3.3.	Trazabilidad	9
3.3.4.	Proceso de cambio	10
3.3.5.	Métricas de volatilidad	10
3.4.	Herramientas CASE para IR	10
3.4.1.	Clasificación de herramientas CASE (upper, lower, integrated)	10
3.4.2.	Funcionalidades esperables en gestión de requisitos	10
3.4.3.	Criterios de selección, limitaciones y costos	11
3.5.	IR en procesos ágiles	11
3.5.1.	Product backlog, historias de usuario y criterios de aceptación	11
3.5.2.	Grooming y Definition of Ready / Definition of Done	11
3.5.3.	BDD y formato Gherkin	11
3.5.4.	Trazabilidad en entornos ágiles	12
3.5.5.	Contraste con la IR documental (tradicional)	12
4.	Correcciones aplicadas	13
5.	Gestión del cambio y CCB	13
5.1.	RFC-01	13
5.2.	RFC-02	13
5.3.	RFC-03	13
5.4.	Acta del CCB	13
5.5.	Versión actualizada del ERS	13
6.	Trazabilidad en herramienta CASE	13
6.1.	Estructura del tablero	13
6.2.	Campos personalizados	13
6.3.	Trazabilidad bidireccional	13
6.4.	Evidencia	13
6.5.	Matriz de trazabilidad	13
7.	Línea base con Git	13
7.1.	Repositorio y estructura de carpetas	13
7.2.	Historial de commits	13
7.3.	Tag anotado de baseline	13
7.4.	CHANGELOG.md	13
7.5.	Capturas del historial	13
8.	Comparativa de herramientas CASE	13
9.	IR tradicional frente a IR ágil	13

10.	Conclusiones críticas	13
11.	Distribución del trabajo	13
12.	Referencias	14
13.	Anexos	14
13.1.	Anexo A	14
13.2.	Anexo B	14
13.3.	Anexo C	14
13.4.	Anexo D	14
13.5.	Anexo E	14
13.6.	Anexo F	14

Índice de figuras

Índice de tablas

1.	Técnicas de validación de requisitos y criterio de selección	8
2.	Fases del proceso de inspección de Fagan [10] y su instanciación en la PE4	8
3.	Roles de la inspección de Fagan y responsabilidad de cada uno	8

1 Resumen

2 Objetivos

2.1 Objetivo general

Aplicar técnicas formales de validación de requisitos, gestionar cambios mediante un Change Control Board (CCB) simulado, implementar la trazabilidad del ERS en una herramienta CASE, establecer una línea base con Git y comparar metodológicamente la Ingeniería de Requisitos (IR) tradicional frente a la IR ágil, sobre el ERS real del proyecto fin de curso MundiPets.

2.2 Objetivos específicos

- Ejecutar una inspección formal tipo Fagan sobre el ERS del PFC, con roles diferenciados, lista de verificación y registro de defectos.
- Calcular e interpretar métricas de inspección (densidad de defectos, distribución por tipo y severidad, tasa de corrección).
- Tramitar tres solicitudes formales de cambio (RFC) con análisis de impacto sustentado en la matriz de trazabilidad.
- Deliberar y decidir en un CCB simulado, dejando acta motivada y versión actualizada del ERS.
- Construir la trazabilidad bidireccional del ERS en una herramienta CASE de gestión (Jira o Trello) con campos personalizados y enlaces verificables.
- Establecer y publicar la línea base del ERS con control de versiones (commit semántico, tag anotado y `CHANGELOG.md`).
- Comparar herramientas CASE de IR y contrastar IR tradicional frente a IR ágil, cerrando con conclusiones críticas propias.

3 Fundamento teórico

3.1 Marco normativo de la validación y la gestión de requisitos

La validación de requisitos no es una actividad discrecional del equipo sino un proceso normado. La ISO/IEC/IEEE 29148:2018 la ubica de forma explícita en su §6.4, dentro del conjunto de procesos de definición de requisitos de stakeholders y de requisitos del sistema, y establece que un requisito debe ser validado para confirmar que expresa la necesidad real del stakeholder, además de verificado para confirmar que está correctamente formulado [1]. La misma norma fija en su §5.2 las propiedades que debe cumplir un requisito individual necesario, apropiado, no ambiguo, completo, singular, factible, verificable, correcto y conforme y las que debe cumplir el conjunto completo, consistente, factible, comprensible, y capaz de ser validado [1]. En esta práctica esas propiedades se agruparon en las seis dimensiones que estructuran el checklist del Anexo A: ambigüedad, completitud, consistencia, verificabilidad, trazabilidad y factibilidad.

El control de cambios sobre los requisitos ya validados pertenece a un proceso distinto. La ISO/IEC/IEEE 15288:2023 lo sitúa en el proceso técnico de gestión de la configuración, que exige identificar los elementos de configuración, establecer líneas base, controlar los cambios sobre ellas y auditar su integridad [2]; la ISO/IEC/IEEE 12207:2017 replica esa estructura en el ciclo de vida del software y añade el proceso de gestión de la información, que es el que obliga a mantener la trazabilidad entre versiones del documento [3]. La consecuencia práctica para MundiPets es directa: una vez que el ERS v3.0 quedó bajo línea base, ningún requisito puede modificarse sin una solicitud formal de cambio, y toda modificación debe generar una nueva versión identificable. Ese es el mecanismo que la Sección 6 instrumenta con los RFC y el CCB.

La calidad del producto que los requisitos describen se evalúa contra el modelo de la ISO/IEC 25010:2023, que define ocho características de calidad del producto software: idoneidad funcional, eficiencia de desempeño, compatibilidad, interacción (usabilidad), fiabilidad, seguridad, mantenibilidad y flexibilidad [4]. En el ERS de MundiPets, cada requisito no funcional se declara asociado a una de estas características, lo que permite comprobar que la cobertura del modelo es completa y detectar características sin ningún RNF asociado. La revisión de esa correspondencia (Tabla 36 del ERS) produjo dos de los defectos registrados en el Anexo 14.2.

Desde la perspectiva del cuerpo de conocimiento de la disciplina, el SWEBOK v4.0 dedica su Área de Conocimiento de Requisitos de Software a estos mismos procesos: la validación de requisitos §1.6, las consideraciones prácticas incluida la gestión del cambio y la trazabilidad §1.7 y las herramientas de ingeniería de requisitos §1.8 [5]. El SWEBOK es explícito en un punto que la práctica suele desatender: la revisión y la inspección son técnicas de validación distintas, y solo la segunda por su carácter formal, su asignación de roles y su registro cuantitativo produce métricas utilizables para decidir si el documento puede pasar a línea base [5]. Pohl desarrolla esa misma distinción y sistematiza las técnicas de validación por perspectivas, que es el enfoque que se adoptó al repartir bloques de propiedades entre inspectores [6].

Por último, la figura del Comité de Control de Cambios (CCB) no proviene de las normas de ingeniería de software sino de la gestión de proyectos. El PMBOK, en su séptima edición, describe el Change Control Board como el grupo formalmente constituido responsable de revisar, evaluar, aprobar, diferir o rechazar los cambios sobre el proyecto, y de registrar esas decisiones [7]. Cuatro elementos de esa definición son los que se replicaron en la Sección 6: composición formal con roles diferenciados, evaluación del impacto antes de decidir, decisión motivada entre cuatro resultados posibles, y registro documental de la deliberación.

3.2 Validación de requisitos

3.2.1 Verificación frente a validación

La distinción es la que fija la 29148:2018 y conviene enunciarla sin ambigüedad porque de ella depende el diseño del checklist. Verificar un requisito es comprobar que está bien construido: que es no ambiguo, singular, verificable y trazable, es decir, que cumple las propiedades formales de §5.2. Validar un requisito es comprobar que describe la necesidad correcta: que el stakeholder que lo originó lo reconoce como suyo y que resolverlo aporta el valor esperado [1]. En la formulación clásica, verificar responde a ¿estamos construyendo el producto correctamente? y validar a ¿estamos construyendo el producto correcto? [5].

La inspección Fagan que se ejecutó en esta práctica es fundamentalmente una técnica de verificación documental, pero produce hallazgos de validación cuando confronta un requisito con su fuente. El defecto D-22 del Anexo 14.2 ilustra el segundo caso: el requisito legal RL-01 declara obligatoria la revocación del consentimiento, ninguna función del sistema la implementa, y el hallazgo solo aparece al recorrer la trazabilidad hacia atrás desde el requisito legal hasta la fuente normativa [8].

3.2.2 Técnicas de validación

La literatura y la norma coinciden en un repertorio de técnicas que se diferencian por su grado de formalidad, el coste que imponen y el tipo de defecto que detectan. La Tabla 1 las resume y justifica la elección de la inspección para esta práctica.

Tabla 1: Técnicas de validación de requisitos y criterio de selección

Técnica	Formalidad	Qué detecta mejor	Limitación principal
Revisión informal	Baja	Errores evidentes de redacción y omisiones gruesas.	Sin roles ni registro: los hallazgos no son reproducibles ni medibles [5].
Walkthrough	Media	Malentendidos sobre el comportamiento esperado; el autor guía la lectura.	El autor conduce la sesión, lo que puede sesgar la detección hacia lo que considera relevante [9].
Inspección de Fagan	Alta	Defectos de consistencia, completitud y verificabilidad; produce métricas de proceso.	Coste alto en horas-persona y necesidad de preparación individual previa [10].
Prototipado	Media	Requisitos mal entendidos o faltantes desde la perspectiva del usuario.	No detecta defectos de especificación en requisitos no visibles en la interfaz [6].
Listas de verificación	Media	Omisiones sistemáticas frente a un estándar de referencia.	Solo encuentra lo que la lista pregunta; puede inducir falsa sensación de cobertura [11].
Lectura por perspectivas	Media-alta	Defectos específicos del punto de vista asignado (usuario, diseñador, probador).	Requiere inspectores con formación en el rol asignado [6].

Se seleccionó la inspección de Fagan combinada con lectura por perspectivas y un checklist derivado de la 29148 porque es la única del repertorio que satisface simultáneamente los tres requisitos del contexto: registro estructurado y auditable de cada defecto, asignación de roles independientes del autor, y generación de métricas de proceso que permiten decidir objetivamente si el documento puede pasar a línea base.

3.2.3 Fases y roles de la inspección de Fagan

Fagan definió el proceso en seis fases y cuatro roles, con una regla que es la que le da su poder y la que más se incumple en la práctica: la reunión detecta defectos y tiene prohibido discutir soluciones [10]. La Tabla 2 recoge las fases y la Tabla 3 los roles, en ambos casos indicando cómo se instanciaron en esta práctica.

Tabla 2: Fases del proceso de inspección de Fagan [10] y su instanciación en la PE4

Fase	Propósito	Instanciación en MundiPets
Planificación	Definir el alcance, seleccionar el material y asignar roles.	Alcance de 52 páginas efectivas; sesión única; 4 roles distribuidos entre 2 integrantes.
Visión general	Presentar el material que será inspeccionado.	Sesión de 20 min a cargo del autor del ERS, según el borrador.
Preparación	Cada inspector examina el material por separado y registra defectos.	4 h por inspectora; Anexo A firmado y con commit fechado previo a la reunión.
Reunión	El lector parafrasea; los inspectores señalan defectos; el moderador modera y registra.	Sesión única de 90 min, dentro del límite establecido.
Retrabajo	Corregir los defectos registrados.	100 % de los críticos y mayores; detalle en la Sección 5.
Seguimiento	Verificar que las correcciones sean efectivas y decidir el cierre.	Verificación antes del tag <code>baseline-v3.1</code> .

Tabla 3: Roles de la inspección de Fagan y responsabilidad de cada uno

Rol	Responsabilidad
Moderador	Planifica la sesión, controla el tiempo, impide la discusión de soluciones, consolida el registro y decide el cierre.
Lector	Parafrasea el material con sus propias palabras; si no puede reformular un requisito sin ambigüedad, existe un problema de interpretación.
Inspector 1 e Inspector 2	Examinan el material durante la preparación desde perspectivas distintas y presentan sus hallazgos en la reunión.
Autor	Aclara dudas de hecho y ejecuta el retrabajo; no debe defender el documento durante la reunión.

Nota: la literatura recomienda que estos roles los ocupen personas distintas. En esta práctica el equipo lo integran dos personas, por lo que cada una asumió dos roles; las medidas adoptadas para preservar el efecto de la separación se detallan en §4.1.

3.2.4 Métricas de inspección

Las métricas de una inspección no son un adorno del informe: son el instrumento con el que se decide si el documento está listo y si el propio proceso de inspección fue eficaz. Las cinco que se calculan en la Sección 4.7 son las que la literatura considera mínimas [9, 10]: la densidad de defectos (defectos por página) estima la calidad del documento y permite comparar entregas sucesivas; la distribución por tipo indica qué propiedad de calidad está más comprometida y, por tanto, qué parte del proceso de especificación hay que corregir; la distribución por severidad determina el volumen de retrabajo obligatorio; la tasa de detección por inspector revela si la preparación fue homogénea o si algún bloque quedó infra-inspeccionado; y el esfuerzo en horas-persona, combinado con el número de defectos, da la eficiencia del proceso y permite estimar el coste de la siguiente inspección.

3.2.5 Caso aplicado

La aplicación industrial documentada por Fagan en IBM sobre módulos de sistema operativo reportó que la inspección detectaba del orden de dos tercios de los defectos antes de la fase de prueba, con una reducción neta del esfuerzo total del proyecto [10]. Gilb y Graham documentan resultados análogos en documentos de requisitos y sostienen que la inspección de especificaciones tiene mejor retorno que la de código, porque un defecto de requisitos se propaga a diseño, código y pruebas [9]. El caso de MundiPets es consistente con esa observación: de los 27 defectos detectados, los cinco críticos afectan a requisitos que ya estaban implementados parcialmente en el MVP, de modo que su detección en la especificación evitó retrabajo aguas abajo.

3.3 Gestión de requisitos

3.3.1 Gestión de la configuración del ERS y línea base

Gestionar requisitos es gestionar la configuración de un artefacto que cambia. La ISO/IEC/IEEE 15288:2023 define línea base como una especificación o producto formalmente revisado y acordado, que sirve de base para el desarrollo posterior y que solo puede cambiarse mediante procedimientos formales de control de cambios [2]. Tres consecuencias operativas se derivan de esa definición y son las que gobiernan la Sección 8: la línea base debe ser identificable (un tag, no una carpeta con fecha), inmutable (no se reescribe, se supersede con una nueva versión) y auditable (el historial debe permitir reconstruir qué cambió, quién lo pidió y quién lo aprobó).

El versionado del ERS de MundiPets sigue el esquema mayor.menor: se incrementa la versión mayor cuando cambia el alcance del documento (de ERS parcial a ERS completa, v2.0 → v3.0) y la menor cuando se aplican correcciones y cambios aprobados sobre el mismo alcance (v3.0 → v3.1). Esa versión debe replicarse idéntica en cuatro lugares: portada del ERS, historial de revisiones, `CHANGELOG.md` y tag de Git; la incoherencia entre ellos es un defecto de gestión de configuración, y de hecho fue uno de los detectados en la inspección (D-04, Anexo B).

3.3.2 Atributos de los requisitos

Un requisito gestionable no es una frase: es un registro con atributos. La 29148:2018 recomienda mantener, como mínimo, identificador, prioridad, estado, fuente, razón de negocio, criterio de verificación y dependencias [1]. El ERS de MundiPets aplica una plantilla uniforme de ocho atributos a los 59 elementos especificados, lo que hizo posible el análisis de impacto de los RFC: sin el atributo fuente no se habría podido reconstruir la trazabilidad hacia atrás de RL-01 hasta la LOPDP [8], y sin el atributo criterio de verificación no se habrían detectado los defectos de verificabilidad D-14, D-19 y D-20.

3.3.3 Trazabilidad

La trazabilidad se clasifica según el momento del ciclo de vida y según la dirección del enlace [5, 6]:

- **Pre-traceability:** enlaza el requisito con su origen: el stakeholder, la entrevista, la evidencia de campo o la norma legal que lo motivó. Responde a ¿por qué existe este requisito? y es la que permite justificar o descartar un cambio.
- **Post-traceability:** enlaza el requisito con los artefactos que lo realizan: caso de uso, clase, módulo, caso de prueba. Responde a ¿qué se rompe si cambia este requisito? y es la que alimenta el análisis de impacto.
- **Hacia atrás (backward) y hacia adelante (forward):** la dirección concreta del recorrido. La trazabilidad es bidireccional cuando todo requisito tiene al menos un enlace en cada sentido; un requisito sin enlace hacia atrás es un requisito sin justificación, y uno sin enlace hacia adelante es un requisito huérfano, candidato a ser eliminado o a revelar una omisión en el diseño [11].

El instrumento operativo es la matriz de trazabilidad, que en esta práctica adopta la forma Stakeholder → RF → CU → Clase/Módulo → Prueba (Tabla 23). Su valor no es documental sino operativo: es el artefacto que se

recorre para responder al análisis de impacto de un RFC, y por eso la Sección 6 deriva cada impacto de una fila concreta de la matriz y no de una estimación a ojo.

3.3.4 Proceso de cambio

El ciclo de vida de un cambio sobre requisitos bajo línea base tiene cinco pasos, y omitir cualquiera de ellos invalida el control [7, 11]:

1. **Solicitud (RFC):** alguien identificable propone el cambio, declara el requisito afectado y da una justificación de negocio.
2. **Análisis de impacto:** se recorre la matriz de trazabilidad para identificar requisitos y artefactos afectados directa e indirectamente, y se estima el coste en horas-persona y el riesgo.
3. **Decisión del CCB:** el comité delibera y resuelve entre cuatro resultados aprobado, aprobado con condiciones, rechazado o diferido, con justificación registrada.
4. **Implementación:** se modifican el requisito y todos los artefactos identificados en el impacto, y se incrementa la versión.
5. **Cierre:** se verifica que la implementación corresponde a lo aprobado y se sella la nueva línea base.

3.3.5 Métricas de volatilidad

La volatilidad de requisitos mide cuánto cambia la especificación por unidad de tiempo y es el indicador temprano de que el alcance no está bajo control [11]. Se expresa habitualmente como:

$$V = \frac{\text{Añadidos} + \text{Modificados} + \text{Eliminados}}{\text{Total de requisitos en la línea base inicial}} \times 100$$

Para MundiPets, los tres RFC aprobados añadieron 2 requisitos (RF-26 y RF-27) y modificaron 7 (RF-01, RF-06, RF-25, RNF-06, RNF-16, RL-01 y RL-02/RL-03 como bloque), sin eliminaciones, sobre una base de 59 elementos:

$$V = \frac{2 + 7 + 0}{59} \times 100 = 15,3 \%$$

En el ciclo de la Unidad IV. Es un valor alto para una especificación que se declaraba completa, y su lectura es la que se recoge en la conclusión C-2 de la Sección 11: la volatilidad no proviene de cambios de mercado sino de defectos de especificación que la inspección puso al descubierto, lo que sugiere que la v3.0 no debió declararse completa sin una validación formal previa.

3.4 Herramientas CASE para IR

3.4.1 Clasificación de herramientas CASE (upper, lower, integrated)

Las herramientas CASE (*Computer-Aided Software Engineering*) automatizan actividades del ciclo de vida del software y se clasifican tradicionalmente según la etapa del desarrollo que soportan. Las herramientas *upper CASE* dan soporte a las fases tempranas análisis y diseño mediante modelado de requisitos, diagramas de flujo de datos y modelos entidad-relación; las *lower CASE* se concentran en las fases posteriores del ciclo de vida, como generación de código, pruebas y mantenimiento; y las integradas (*I-CASE*) cubren el ciclo completo, desde la captura de requisitos hasta el despliegue, mediante un repositorio compartido que garantiza consistencia entre artefactos [1]. Esta clasificación es coherente con el proceso de gestión de requisitos descrito en IEEE/ISO/IEC 15288:2023, donde la trazabilidad entre fases exige que el repositorio de la herramienta upper CASE permanezca sincronizado con los artefactos de diseño e implementación gestionados aguas abajo.

En el contexto específico de la Ingeniería de Requisitos, las herramientas de gestión de requisitos (RM tools) como Jira, IBM DOORS o Jama Connect operan como CASE de tipo upper o integrado, dado que combinan captura, atributos, trazabilidad y control de versiones de los requisitos; de hecho, una encuesta reciente a 84 profesionales de la industria confirma que este tipo de herramientas concentra el mayor uso reportado en la práctica, aunque persiste una baja adopción de tecnologías más avanzadas de apoyo a la IR [12].

3.4.2 Funcionalidades esperables en gestión de requisitos

De acuerdo con el proceso de gestión de requisitos establecido en IEEE/ISO/IEC 29148:2018 [1], una herramienta CASE orientada a IR debe ofrecer como mínimo: (a) un repositorio centralizado de requisitos con control de acceso; (b) atributos configurables por requisito (prioridad, fuente, estado, criticidad); (c) trazabilidad bidireccional, hacia atrás (stakeholder/documento origen) y hacia adelante (diseño, código, prueba); (d) *baselining* o creación de líneas base versionadas; (e) gestión de cambios integrada con un flujo de aprobación tipo CCB; (f) colaboración

multiusuario en tiempo real; y (g) integración con sistemas de control de versiones (VCS) como Git, lo que permite correlacionar el historial de commits con la evolución del requisito. Esta última funcionalidad es la que en PE4 conecta directamente la herramienta CASE elegida (Jira o Trello) con el baseline de Git de la Sección 8 del informe.

Un estudio reciente sobre metodologías de introducción de trazabilidad en equipos ágiles confirma empíricamente que la ausencia de una de estas funcionalidades particularmente la trazabilidad bidireccional configurada desde el inicio del proyecto es la causa más común de que los enlaces entre requisitos y artefactos se degraden con el tiempo, incluso cuando la herramienta CASE los soporta técnicamente [13].

3.4.3 Criterios de selección, limitaciones y costos

La selección de una herramienta CASE debe basarse en criterios objetivos: madurez de la organización, tamaño del equipo, complejidad regulatoria del dominio, curva de aprendizaje, costo de licenciamiento (modelo por usuario o por proyecto) y capacidad de integración con el resto del stack de desarrollo. Herramientas como IBM DOORS o Jama Connect ofrecen trazabilidad y auditoría robustas orientadas a sectores regulados (aeroespacial, automotriz, salud), a costa de una curva de aprendizaje alta y licenciamiento costoso; herramientas como Jira o Trello, en cambio, priorizan la agilidad y el bajo costo (incluso gratuito para equipos pequeños), sacrificando funcionalidades avanzadas de auditoría y cumplimiento normativo nativo. Esta tensión entre robustez de trazabilidad y costo/curva de aprendizaje coincide con lo reportado empíricamente: el costo y el esfuerzo de adopción de la herramienta figuran entre los principales obstáculos que los profesionales identifican al momento de seleccionar y adoptar tecnología de apoyo a la IR [12]. Este eje se retoma en la Sección 9 (comparativa de herramientas CASE) de este informe.

3.5 IR en procesos ágiles

3.5.1 Product backlog, historias de usuario y criterios de aceptación

En el desarrollo ágil, el *product backlog* reemplaza al documento de especificación formal como artefacto vivo de requisitos, priorizado y refinado continuamente. La historia de usuario (*user story*) es la unidad semi-estructurada más usada para capturar requisitos en este contexto: un texto breve, típicamente en formato “Como [rol], quiero [funcionalidad], para [beneficio]”, acompañado de sus criterios de aceptación [14]. Una revisión sistemática de la literatura sobre investigación en historias de usuario identifica que, pese a su simplicidad aparente, la ambigüedad y la falta de verificabilidad siguen siendo los defectos de calidad más frecuentes reportados en estudios empíricos, lo cual motiva el uso de técnicas de procesamiento de lenguaje natural y de checklists de calidad (el framework INVEST, entre otros) para su validación antes de ingresar al sprint [14, 15].

3.5.2 Grooming y Definition of Ready / Definition of Done

El *backlog grooming* (o *refinement*) es la actividad continua mediante la cual el equipo discute, estima y detalla los elementos del backlog para que alcancen un estado de preparación suficiente antes del *sprint planning*. Durante esta actividad se estima habitualmente el esfuerzo de cada elemento mediante *story points*; un estudio empírico reciente sobre proyectos ágiles de código abierto documenta que estas estimaciones cambian con frecuencia después de la planificación del sprint, lo cual constituye una fuente medible de variabilidad que el grooming continuo busca minimizar [16]. Dos convenciones ampliamente adoptadas por los equipos Scrum formalizan este umbral de calidad: la *Definition of Ready* (DoR), un checklist que define cuándo una historia está suficientemente clara, estimada y con criterios de aceptación definidos para entrar a un sprint; y la *Definition of Done* (DoD), que define cuándo un incremento se considera terminado y potencialmente entregable. A diferencia de la DoD, la DoR no forma parte oficial de la Guía de Scrum, pero es una práctica de facto ampliamente reportada como mecanismo de control de calidad de requisitos en el flujo ágil.

3.5.3 BDD y formato Gherkin

El *Behavior-Driven Development* (BDD) extiende la historia de usuario con escenarios de comportamiento escritos en lenguaje natural estructurado el formato Gherkin, basado en las palabras clave *Given-When-Then* que son a la vez comprensibles para los interesados de negocio y ejecutables como pruebas de aceptación automatizadas [17]. Una revisión sistemática reciente sobre BDD, que analizó 31 estudios primarios, concluye que su principal aporte a la IR es reducir la brecha de comunicación entre stakeholders técnicos y no técnicos, al anclar cada criterio de aceptación a un escenario verificable de forma automática, lo que en la práctica convierte la especificación de requisitos en una suite de pruebas viva [17]. Esta característica conecta directamente con la trazabilidad: cada escenario Gherkin actúa como un enlace ejecutable entre el requisito y su verificación.

3.5.4 Trazabilidad en entornos ágiles

La trazabilidad en proyectos ágiles enfrenta un desafío estructural: la naturaleza informal y cambiante de las historias de usuario dificulta mantener enlaces estables hacia atrás (stakeholder) y hacia adelante (código, prueba) sin una estrategia explícita [13]. El estudio de Maro et al., que evaluó una metodología de introducción de trazabilidad (TraciMo) en un equipo ágil real del dominio financiero, encontró que la trazabilidad no emerge espontáneamente del uso de un tablero Kanban o Scrum: requiere definir explícitamente un modelo de información de trazabilidad (qué se enlaza con qué) desde el inicio del proyecto, y que dicho modelo sea mantenido activamente durante el refinamiento del backlog, no solo durante la planificación inicial [13]. Este hallazgo es directamente aplicable a la Sección 7 del informe (trazabilidad en Jira/Trello), pues confirma la necesidad de reportar explícitamente los requisitos huérfanos exigidos por la rúbrica.

3.5.5 Contraste con la IR documental (tradicional)

Frente a la IR tradicional (documento ERS único, validado en un hito formal, línea base rígida), la IR ágil distribuye la validación en ciclos cortos e iterativos: el criterio de aceptación reemplaza a la revisión formal exhaustiva, y el backlog reemplaza a la matriz de trazabilidad estática [18]. Esto no elimina la necesidad de trazabilidad ni de control de cambios como demuestra el propio desarrollo del presente trabajo, al exigir una matriz de trazabilidad y un CCB simulado sobre un ERS documental sino que reubica dónde y cuándo ocurre esa disciplina: en el enfoque documental ocurre de forma centralizada y previa al desarrollo (inspección Fagan, CCB); en el enfoque ágil ocurre de forma distribuida y continua (refinamiento, DoR/DoD, pruebas BDD) [18]. Esta comparación se desarrolla con mayor detalle, en forma de tabla de seis dimensiones, en la Sección 10 del informe.

4 Correcciones aplicadas

5 Gestión del cambio y CCB

5.1 RFC-01

5.2 RFC-02

5.3 RFC-03

5.4 Acta del CCB

5.5 Versión actualizada del ERS

6 Trazabilidad en herramienta CASE

6.1 Estructura del tablero

6.2 Campos personalizados

6.3 Trazabilidad bidireccional

6.4 Evidencia

6.5 Matriz de trazabilidad

7 Línea base con Git

7.1 Repositorio y estructura de carpetas

7.2 Historial de commits

7.3 Tag anotado de baseline

7.4 CHANGELOG.md

7.5 Capturas del historial

8 Comparativa de herramientas CASE

9 IR tradicional frente a IR ágil

10 Conclusiones críticas

11 Distribución del trabajo

12 Referencias

- [1] International Organization for Standardization, «ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering,» International Organization for Standardization, Geneva, Switzerland, inf. téc., 2018. DOI: 10.1109/IEEESTD.2018.8559686
- [2] International Organization for Standardization, «ISO/IEC/IEEE 15288:2023 – Systems and software engineering – System life cycle processes,» International Organization for Standardization, Geneva, Switzerland, inf. téc., 2023.
- [3] International Organization for Standardization, «ISO/IEC/IEEE 12207:2017 – Systems and software engineering – Software life cycle processes,» International Organization for Standardization, Geneva, Switzerland, inf. téc., 2017.
- [4] International Organization for Standardization, «ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model,» International Organization for Standardization, Geneva, Switzerland, inf. téc., 2023.
- [5] H. Washizaki, ed., *SWEBOK Guide to the Software Engineering Body of Knowledge, Version 4.0*, 4.0. Los Alamitos, CA, USA: IEEE Computer Society, 2024.
- [6] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Berlin, Germany: Springer, 2010.
- [7] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th. Newtown Square, PA, USA: Project Management Institute, 2021.
- [8] Asamblea Nacional del Ecuador, *Ley Orgánica de Protección de Datos Personales*, Registro Oficial Suplemento 459, Quito, Ecuador, 2021.
- [9] T. Gilb y D. Graham, *Software Inspection*. Wokingham, UK: Addison-Wesley, 1993.
- [10] M. E. Fagan, «Design and code inspections to reduce errors in program development,» *IBM Systems Journal*, vol. 15, n.º 3, págs. 182-211, 1976. DOI: 10.1147/sj.153.0182
- [11] K. E. Wiegers y J. Beatty, *Software Requirements*, 3rd. Redmond, WA, USA: Microsoft Press, 2013.
- [12] M. Ozkaya, D. Akdur, E. C. Toptani, B. Kocak y G. Kardas, «Practitioners’ Perspectives towards Requirements Engineering: A Survey,» *Systems*, vol. 11, n.º 2, pág. 65, 2023. DOI: 10.3390/systems11020065
- [13] S. Maro, J.-P. Steghöfer, P. Bozzelli y H. Muccini, «TraciMo: A Traceability Introduction Methodology and Its Evaluation in an Agile Development Team,» *Requirements Engineering*, vol. 27, n.º 1, págs. 53-81, 2022. DOI: 10.1007/s00766-021-00361-5
- [14] I. K. Raharjana, D. Siahaan y C. Fatichah, «User Stories and Natural Language Processing: A Systematic Literature Review,» *IEEE Access*, vol. 9, págs. 53 811-53 826, 2021. DOI: 10.1109/ACCESS.2021.3070606
- [15] A. R. Amna y G. Poels, «Systematic Literature Mapping of User Story Research,» *IEEE Access*, vol. 10, págs. 51 723-51 746, 2022. DOI: 10.1109/ACCESS.2022.3173745
- [16] J. Pasuksmit, P. Thongtanunam y S. Karunasekera, «Story Points Changes in Agile Iterative Development: An Empirical Study and a Prediction Approach,» *Empirical Software Engineering*, vol. 27, n.º 6, pág. 156, 2022. DOI: 10.1007/s10664-022-10192-9
- [17] M. S. Farooq, U. Omer, A. Ramzan, M. A. Rasheed y Z. Atal, «Behavior Driven Development: A Systematic Literature Review,» *IEEE Access*, vol. 11, págs. 88 008-88 024, 2023. DOI: 10.1109/ACCESS.2023.3302356
- [18] Z. Hoy y M. Xu, «Agile Software Requirements Engineering Challenges-Solutions: A Conceptual Framework from Systematic Literature Review,» *Information*, vol. 14, n.º 6, pág. 322, 2023. DOI: 10.3390/info14060322

13 Anexos

13.1 Anexo A

13.2 Anexo B

13.3 Anexo C

13.4 Anexo D

13.5 Anexo E

13.6 Anexo F